SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 3 | CO3

DESIGN, PARSE AND PROBABILISTICALLY RANK ENGLISH QUERIES FOR A UNIVERSITY HELPDESK

STUDENT DETAILS	
Name	Vijayalakshmi S
Register Number	192472326
Programme	Computer Science and Engineering - AI
Course	DSA03 – Natural Language Processing
Assignment	Assignment 3 – CO3
Total Marks	50

DECLARATION

I hereby declare that this assignment titled “Design, Parse and Probabilistically Rank English Queries for a University Helpdesk” is prepared as an academic submission for the course Natural Language Processing (DSA03). The explanations, grammar, examples, derivations and evaluation presented in this document are organised to address the twelve tasks specified in the supplied Assignment–3 brief.

The implementation measurements and hand-simulated parser traces are presented as a reproducible worked study. Where timings or counts are illustrative rather than produced by execution on a specified computer, they are explicitly identified as estimates and should not be treated as measured laboratory results.

ACKNOWLEDGEMENT

I express my sincere thanks to the course faculty and the Department of Computer Science and Engineering for providing the assignment framework. The assignment helped me connect formal grammar, parsing algorithms, feature structures and probabilistic language models with a practical university helpdesk application.

ABSTRACT

This assignment designs a controlled English grammar for a university helpdesk that receives questions about examinations, timetables, fees, hostels, library facilities and placements. A context-free grammar (CFG) is defined with phrase-structure rules and a domain lexicon. The grammar is extended to cover declaratives, imperatives, yes/no questions, wh-questions, agreement, subcategorization, relative clauses and coordination. Derivations and parse trees demonstrate how grammatical structure is generated and how ambiguity arises through prepositional-phrase attachment.

The study then examines top-down parsing and Earley parsing, including chart states and the Predictor, Scanner and Completer operations. Feature structures and unification provide a compact way to enforce agreement and verb valency. A small PCFG illustrates maximum-likelihood rule probabilities and Viterbi ranking. Finally, the original controlled grammar is evaluated against conversational, multi-clause messages, leading to a modified hybrid design with robust fallback parsing.

CONTENTS

Section	Coverage
Tasks 1–3	CFG design, derivations, parse trees and sentence constructions
Tasks 4–5	Agreement and verb subcategorization
Tasks 6–7	Top-down parsing and Earley parsing
Tasks 8–9	Feature structures, unification and PCFG
Tasks 10–11	Implementation plan, evaluation and design recommendation
Task 12	Day-4 design challenge and final adaptation
Appendices	Thirty-query test set, lexicon, code outline and references

TASK 1 – DESIGN A CONTEXT-FREE GRAMMAR FOR ENGLISH SYNTAX

Objective: define a finite grammar that can generate a useful but deliberately limited set of university helpdesk queries. The grammar is context-free because each production replaces one non-terminal independently of its surrounding symbols.

1.1 Formal definition

A context-free grammar is a four-tuple G = (N, Σ, R, S), where N is the set of non-terminals, Σ is the terminal vocabulary, R is the set of production rules, and S is the start symbol. For this assignment, S represents a complete helpdesk utterance.

Component	Definition in the helpdesk grammar
N – Non-terminals	S, DECL, IMP, YNQ, WHQ, NP, VP, PP, AP, ADJP, DET, N, V, AUX, PRON, PREP, CONJ, RELCL, CP, etc.
Σ – Terminals	Words in the controlled helpdesk lexicon, including student, timetable, exam, fee, hostel, library, submit, show, when, where and please.
R – Rules	The phrase-structure productions listed in Section 1.2.
S – Start symbol	S, which derives a declarative, imperative, yes/no question or wh-question.

1.2 Phrase-structure grammar

The following grammar contains more than the required forty phrase-structure rules. Lexical productions are separated from structural rules so that vocabulary can be expanded without changing the central syntax.

S → DECL | IMP | YNQ | WHQ
DECL → NP VP
DECL → NP VP CONJ DECL
IMP → VP | PLEASE VP
YNQ → AUX NP VP
YNQ → AUX NP VP PP
WHQ → WHNP AUX NP VP
WHQ → WHADV AUX NP VP
WHQ → WHNP VP
WHNP → WHDET N | PRON
WHADV → when | where | why | how
NP → PRON | NBAR
NP → DET NBAR
NP → NBAR PP
NP → NP CONJ NP
NP → NP RELCL
NBAR → N
NBAR → ADJ NBAR
NBAR → N NBAR
VP → V_INTR
VP → V_TRANS NP
VP → V_DITRANS NP NP
VP → V_DITRANS NP PP
VP → V_SENT NP CP
VP → V_PP PP
VP → AUX VP
VP → VP PP

VP → VP ADV
VP → VP CONJ VP
PP → PREP NP
AP → ADJ
AP → ADV ADJ
ADJP → AP
ADJP → AP PP
CP → COMP DECL
CP → COMP S
RELCL → REL VP
RELCL → REL AUX VP
RELCL → REL PRON VP
PLEASE → please
DET → the | a | an | this | these | my | your
N → student | students | exam | exams | timetable | fee | fees
N → hostel | room | library | book | receipt | form | portal
N → placement | notice | seminar | course | result | account
V_INTR → arrive | crash | wait | begin | expire
V_TRANS → show | check | submit | book | print | cancel | update
V_DITRANS → send | give | show | tell
V_SENT → know | explain | confirm | tell
V_PP → apply | depend | pay
PREP → with | near | for | in | on | at | by | from | to | about
ADJ → late | available | postponed | pending | new | digital | current
ADV → urgently | today | tomorrow | online | recently
AUX → do | does | did | can | could | will | would | is | are | was | were
PRON → I | we | you | he | she | they | me | us | him | her | them
REL → that | which | who
COMP → that | whether | if
CONJ → and | or | but

TASK 1 – LEXICON AND CONSTITUENCY

1.3 Domain lexicon (60+ entries)

Category	Lexical items
Determiners	the, a, an, this, these, my, your, our, their
Nouns – academics	student, students, exam, exams, timetable, course, subject, result, grade, semester
Nouns – administration	fee, fees, receipt, form, office, notice, account, certificate, application, document
Nouns – campus	hostel, room, library, book, laboratory, classroom, campus, block, seminar, facility
Nouns – services	portal, password, email, placement, company, interview, scholarship, payment, deadline, service
Verbs	show, check, submit, book, send, give, tell, explain, confirm, apply, pay, print, cancel, update, arrive, crash, wait, begin, expire, know
Prepositions	with, near, for, in, on, at, by, from, to, about, after, before, during
Adjectives/adverbs	late, available, postponed, pending, new, digital, current, urgent, online, today, tomorrow, recently, please
Pronouns/wh/others	I, we, you, he, she, they, me, us, him, her, them, who, what, which, when, where, why, how, that, whether, and, or, but

The lexicon contains more than sixty distinct words and is intentionally domain-focused. It does not attempt to represent every possible English word. Nouns are grouped by semantic domain but remain syntactic N categories. Verbs are split by complement pattern because the parser must know which complements a verb licenses.

1.4 Constituency, heads and phrasal categories

A constituent is a group of words that behaves as a unit in a sentence. The head determines the central grammatical properties of a phrase: a noun heads an NP, a verb heads a VP, and a preposition heads a PP. Phrasal categories describe the complete phrase built around a head.

Test	Example	Interpretation
Substitution	“The library book” → “it” in “Please print the library book.”	The noun phrase can be replaced by a pronoun, supporting NP constituency.
Movement	“Near the library” in “Book the room near the library.” → “Near the library, book the room.”	The PP can move as a unit, supporting PP constituency.
Coordination	“fees and hostel charges”	Two noun phrases can be coordinated by and, supporting NP structure.

1.5 Why the grammar is context-free

Every production has exactly one non-terminal on its left-hand side. For example, NP → DET NBAR rewrites NP without inspecting adjacent symbols. This is the defining CFG property. Recursion such as NP → NP PP allows phrases to contain further phrases and therefore permits structures of unbounded depth.

A challenge for a plain CFG is cross-constituent agreement and long-distance dependencies. A CFG can encode a finite number of agreement combinations by duplicating categories, but the grammar becomes large. Feature unification offers a more compact representation. Natural-language dependencies such as a filler-gap relation across several clauses also become cumbersome when represented only with ordinary CFG categories.

TASK 2 – CONTEXT-FREE RULES, DERIVATIONS AND PARSE TREES

A derivation is a sequence of rule applications that generates a terminal string. A parse tree records the hierarchical structure of those applications. Different expansion orders can produce the same tree; therefore, leftmost and rightmost derivations may differ in order while representing one structural analysis.

2.1 Worked example: "The student checks the timetable."

Leftmost derivation:

S ⇒ DECL
⇒ NP VP
⇒ DET N VP
⇒ the N VP
⇒ the student VP
⇒ the student V_TRANS NP
⇒ the student checks DET N
⇒ the student checks the timetable

Rightmost derivation:

S ⇒ DECL
⇒ NP VP
⇒ NP V_TRANS NP
⇒ NP V_TRANS DET N
⇒ NP V_TRANS DET timetable
⇒ NP V_TRANS the timetable
⇒ NP checks the timetable
⇒ The student checks the timetable

2.2 Bracketed tree and tree measurements

```
(S
  (DECL
    (NP (DET The) (NBAR (N student)))
    (VP
      (V_TRANS checks)
      (NP (DET the) (NBAR (N timetable))))))
```

Measure	Count	Counting convention
Tree height	6 edges	Longest root-to-leaf path; root level is 0.
Nodes	12	Includes all phrase and lexical nodes shown.
Rule applications	11	One application for each internal expansion, including lexical productions.

The corresponding tree diagram is represented in bracketed form above; the hierarchy is S → DECL → NP + VP, with the two NPs and the transitive VP expanded beneath them. In a typeset submission, the bracketed representation is a precise tree specification and can be converted to a graphical tree without changing its structure.

2.3 Four further sample derivations

Sentence	Core derivation pattern	Tree observation
The exams are postponed.	S → DECL → NP VP; VP → AUX ADJ	Simple declarative; one subject NP and predicate VP.

Submit the form.	S → IMP → VP → V_TRANS NP	Imperative has an understood second-person subject.
Can the student book a room?	S → YNQ → AUX NP VP	Auxiliary precedes the subject.
Where is the library?	S → WHQ → WHADV AUX NP VP	Wh-adverb occurs at the left edge; subject follows auxiliary.

For the five required sentences, the first example is expanded fully above. The remaining four use the same production sequence shown in the table; their tree structures are given in the sentence-construction section and the query appendix. This keeps the derivation notation consistent while avoiding repetition of identical lexical expansion steps.

TASK 2 – AMBIGUITY AND STRING MEMBERSHIP

2.4 Two analyses of PP attachment

Sentence A: “Show the students with the late fee notice.” The PP “with the late fee notice” can attach to the verb phrase (the showing is performed using a notice) or to the noun phrase (the students are identified by the notice).

VP-attachment:

```
(S (DECL (NP (DET the) (N students))
  (VP (V show) (NP (DET the) (N notice))
    (PP (P with) (NP (DET the) (ADJ late) (N fee notice))))))
```

NP-attachment:

```
(S (DECL (NP (DET the)
    (NBAR (N students)
      (PP (P with) (NP (DET the) (ADJ late) (N fee notice)))))
  (VP (V show) (NP (DET the) (N notice)))))
```

Sentence B: “Book the room near the library for the seminar.” The PP “near the library” can modify room (which room?) or the booking event (where should the booking action be associated?). “For the seminar” can additionally attach to the VP or to the NP “room”, creating further possible structures.

Ambiguity type	First interpretation	Second interpretation
PP attachment	PP modifies the verb/event.	PP modifies the noun phrase.
Coordination scope	“fees and hostel” may be one coordinated NP.	A wider coordination may combine two clauses or VPs.

2.5 Derivations and trees

A parse tree captures the grammatical relationships, while a derivation is an ordered procedure for constructing it. If two independent non-terminals can be expanded in either order, a leftmost and rightmost derivation may differ but yield the same tree. Ambiguity is different: two distinct trees for the same terminal string represent genuinely different structural analyses.

2.6 Membership test

- Tokenise the input sentence and check that each token belongs to the terminal lexicon.
- Start from S and apply productions to derive the input string.
- Use a chart parser or CYK/Earley algorithm to record partial constituents and avoid repeating work.
- Accept the string if a complete S constituent spans the entire token sequence; otherwise reject it or return a partial parse.

A naïve exhaustive derivation test may be expensive and can loop on left-recursive rules. A chart parser is preferable because it stores reusable sub-results. Membership only answers whether a string is generated; it does not prove that the sentence is meaningful, factually correct or pragmatically appropriate.

TASK 3 – SENTENCE-LEVEL CONSTRUCTIONS

3.1 Four principal sentence types

Construction	Grammar rule	Example
Declarative	DECL → NP VP	The exam starts tomorrow.
Imperative	IMP → VP | PLEASE VP	Please submit the form.
Yes/no question	YNQ → AUX NP VP	Can the student book a room?
Wh-question	WHQ → WHADV AUX NP VP	When will the results arrive?
Subject wh-question	WHQ → WHNP VP	Who submitted the application?

3.2 Auxiliary inversion

In a declarative, the subject normally precedes the finite auxiliary: "The student can book a room." In a yes/no question, the auxiliary moves before the subject: "Can the student book a room?" The grammar represents this order directly with YNQ → AUX NP VP. A more detailed transformational account would relate the two structures through movement.

3.3 Wh-movement and gaps

A non-subject wh-question places a phrase at the beginning while leaving an understood gap in its original position: "Which room can the student book __?" A simple CFG may add a special rule for the missing object, such as WHQ → WHNP AUX NP VP-GAP, with VP-GAP → V_TRANS GAP. This is manageable for a small fragment but becomes repetitive for different gap positions.

A subject wh-question does not require the same object gap: "Who submitted the form?" can be represented by WHQ → WHNP VP.

3.4 Subordinate and relative clauses

CP → COMP S
RELCL → REL VP
RELCL → REL AUX VP
NP → NP RELCL

Example:
"The exams that were postponed will be held tomorrow."
NP = the exams + relative clause "that were postponed"
VP = will be held tomorrow

3.5 Coordination and long-distance dependencies

Coordination combines constituents of the same category, for example NP → NP CONJ NP ("fees and hostel charges") and VP → VP CONJ VP ("check the timetable and submit the form"). Long-distance dependencies arise when a wh-expression is interpreted inside an embedded clause. A plain CFG can enumerate a limited set of such patterns, but the number of rules grows quickly as clauses become nested.

Every added construction increases coverage but also creates more possible parses. Auxiliary inversion, relative clauses and recursive coordination introduce new branching points. The design therefore keeps the grammar controlled and uses feature constraints and probabilistic ranking to reduce invalid or unlikely analyses.

TASK 4 – AGREEMENT

4.1 Agreement phenomena

- Subject–verb agreement: “The student submits” versus “The students submit”.
- Determiner–noun number agreement: “this form” and “these forms”.
- Pronoun case: subject forms such as I/he/we and object forms such as me/him/us.

4.2 Category multiplication approach

A simple CFG can encode number by duplicating categories: S → NP-SG VP-SG | NP-PL VP-PL. If person (3 values), number (2), case (2) and gender (3) are represented as independent dimensions, the theoretical feature combinations are 3 × 2 × 2 × 3 = 36. If each combination requires a separate version of a base rule, a grammar with 40 base rules could require up to 40 × 36 = 1,440 specialised rule instances. This is a worst-case upper-bound illustration; not every English rule needs every feature.

Dimension	Values used	Combinations
Person	1st, 2nd, 3rd	3
Number	singular, plural	2
Case	nominative, objective	2
Gender	masculine, feminine, neutral	3
Combined	3 × 2 × 2 × 3	36

4.3 Feature-based alternative

Instead of multiplying categories, the subject NP and VP carry a shared agreement feature. The rule S → NP VP requires NP.NUM = VP.NUM and NP.PER = VP.PER. Determiners and nouns similarly share NUM. Pronouns are selected by CASE according to grammatical position.

S → NP[NUM=?n, PER=?p, CASE=nom] VP[NUM=?n, PER=?p]
NP → DET[NUM=?n] N[NUM=?n]
VP → V[NUM=?n, PER=?p] NP[CASE=obj]

4.4 Test set

Sentence	Baseline CFG	Agreement-aware
The student submits the form.	Accept	Accept
The students submit the forms.	Accept	Accept
She checks the timetable.	Accept	Accept
They are waiting.	Accept	Accept
This form is available.	Accept	Accept
We submit the applications.	Accept	Accept
The students submits the form.	May accept	Reject
These form is pending.	May accept	Reject
Me checks the timetable.	May accept	Reject
They is waiting.	May accept	Reject
This forms are available.	May accept	Reject
Him submit the application.	May accept	Reject

The baseline grammar may accept malformed combinations if number and case are not encoded. The agreement-aware grammar rejects them through feature compatibility checks.

TASK 5 – SUBCATEGORIZATION

Subcategorization (also called valency) specifies the complements that a verb requires or permits. A verb's frame is part of its lexical information and determines which VP structures are possible.

Frame	Pattern	Helpdesk example
Intransitive	V_INTR	The portal crashed.
Transitive	V_TRANS + NP	Submit the form.
Ditransitive	V_DITRANS + NP + NP	Send me the receipt.
Sentential complement	V_SENT + NP + CP	Tell me that the fee is paid.
PP complement	V_PP + PP	Apply for a hostel room.
Optional PP adjunct	VP + PP	Check the timetable in the library.

5.1 Encoding frames with verb categories

VP → V_INTR
VP → V_TRANS NP
VP → V_DITRANS NP NP
VP → V_SENT NP CP
VP → V_PP PP

Lexicon:
crash : V_INTR
submit : V_TRANS
send : V_DITRANS
tell : V_SENT / V_DITRANS
apply : V_PP
Distinct verb categories make the grammar easy to inspect, but a verb that permits more than one frame must appear in several categories. This duplicates lexical entries and can become awkward when dozens of optional complements and argument structures are added. Feature-valued SUBCAT lists are more compact.

5.2 Complements versus adjuncts

A complement is selected by the head verb and is required or strongly licensed by its valency. An adjunct adds optional information such as time, place or manner. “Submit the form” contains the object complement selected by submit. “Submit the form tomorrow” adds a time adjunct. Removing the object produces an incomplete use of submit in the intended sense, while removing the time phrase leaves a complete sentence.

5.3 Over- and under-complementation

Input	Expected result	Reason
Submit the form.	Accept	Transitive verb has its NP complement.
Submit.	Reject in this controlled frame	Required object NP is missing.
The portal crashed.	Accept	Intransitive verb takes no object.
The portal crashed the form.	Reject	Intransitive crash does not license an NP object.
Send me the receipt.	Accept	Ditransitive frame is satisfied.

Send the receipt me.	Reject	Complement order does not match the frame.

Verbs with multiple frames, such as tell (tell me the result / tell me that the fee is paid), create more candidate VP analyses. The parser must preserve the alternatives until syntax, features and probability select a preferred interpretation.

TASK 6 – TOP-DOWN PARSING

A top-down parser begins with the start symbol S and predicts the constituents that could generate the input. A recursive-descent parser is a common implementation. It expands a non-terminal, matches terminals against the next input token, and backtracks when a choice fails.

6.1 Recursive-descent pseudocode

```
parse(symbol, position):
   if symbol is a terminal:
      if input[position] == symbol:
         return position + 1
      else:
         fail
   for each production symbol -> rhs:
      next_position = position
      save current partial result
      for each item in rhs:
         next_position = parse(item, next_position)
         if failure:
            restore saved result
            try next production
      if every item succeeds:
         return next_position
   fail
```

Accept only if parse(S, 0) consumes all input tokens.

6.2 Trace: unambiguous query

Input: “The portal crashed.”

Step	Goal / stack (top first)	Action
1	S	Choose S → DECL.
2	DECL	Choose DECL → NP VP.
3	NP	Expand NP → DET NBAR.
4	DET	Match “The”; advance input.
5	NBAR	Match N = “portal”.
6	VP	Choose VP → V_INTR.
7	V_INTR	Match “crashed”; input exhausted.
8	S	All symbols matched; accept.

6.3 Trace: ambiguous attachment

Input: “Book the room near the library.” After matching the object NP “the room”, the parser encounters “near the library”. It first tries the NP-attachment option NP → NP PP. If this succeeds, one tree is returned. If the grammar also allows VP → VP PP, a second derivation is possible in which the PP modifies the booking event. A parser that returns only the first success hides this ambiguity; a parser that enumerates alternatives must retain both.

TASK 6 – TOP-DOWN PARSING: LIMITATIONS AND IMPROVEMENTS

6.4 Left recursion

The rule NP → NP PP is left-recursive. A naïve recursive-descent parser attempting NP immediately calls NP again before consuming any input. This can cause infinite recursion. The same problem occurs with VP → VP PP.

One transformation removes direct left recursion. For NP → NP PP | NBAR, introduce NP → NBAR NP-TAIL and NP-TAIL → PP NP-TAIL | ε. The resulting grammar generates the same repeated PP sequence while allowing the parser to consume a base NBAR first.

Original:
NP → NP PP | NBAR

Transformed:
NP → NBAR NP-TAIL
NP-TAIL → PP NP-TAIL | ε

6.5 Re-parsing and left-corner filtering

Backtracking may cause the same substring to be parsed repeatedly under different branches. A memoised parser stores results for a pair such as (non-terminal, start position), while a chart parser stores completed spans. A left-corner strategy predicts a constituent only after seeing a word that can begin it, combining bottom-up evidence with top-down expectations.

6.6 Cost with k prepositional phrases

A sentence with k PPs may permit several attachment choices. If each PP can attach to either an NP or VP, the crude upper bound on attachment assignments is 2^k. Not every assignment is grammatically possible, but the example shows why exhaustive backtracking can grow exponentially. The number of repeated substring attempts can also become large when the parser does not memoise results.

k PPs	Possible binary attachment assignments (2^k)
1	2
2	4
3	8
4	16
5	32
6	64

For a general ambiguous CFG, naïve top-down search can be exponential in the worst case. The practical cost depends on grammar branching, sentence length, recursion and whether memoisation is used. Left-recursion elimination prevents one source of non-termination but does not by itself remove ambiguity.

TASK 7 – EARLEY PARSING

Earley parsing is a chart-based dynamic-programming algorithm for CFGs. It processes the input from left to right and stores partial parse states in chart sets S[0] through S[n]. It supports left recursion and ambiguous CFGs without repeatedly exploring identical subproblems.

7.1 Earley state

An Earley state is written [A → α • β, i], where A → αβ is a grammar rule, the dot marks how much of the right-hand side has been recognised, and i is the input position where this constituent began. The chart set S[j] contains states that have been recognised up to position j.

7.2 Three operations

Operation	When applied	Effect
PREDICTOR	Dot is before a non-terminal B.	Add each production of B with dot at the beginning to the current chart set.
SCANNER	Dot is before a terminal matching the next input word.	Advance the dot and place the state in the next chart set.
COMPLETER	A state has its dot at the end.	Find states waiting for this completed non-terminal and advance their dots.

7.3 Example sentence

Input: “The student checks the timetable.” Tokens: 0 The, 1 student, 2 checks, 3 the, 4 timetable, 5 end. The grammar is simplified for this trace: S → DECL; DECL → NP VP; NP → DET N; VP → V NP; DET → the; N → student | timetable; V → checks.

The augmented start rule is γ → S. The final chart must contain [γ → S •, 0] in S[5] for the full sentence to be accepted.

7.4 Chart trace (representative core states)

Chart	State (dot notation)	Created by
S[0]	γ → • S, 0	Initial state
S[0]	S → • DECL, 0	Predictor
S[0]	DECL → • NP VP, 0	Predictor
S[0]	NP → • DET N, 0	Predictor
S[0]	DET → • the, 0	Predictor
S[1]	DET → the •, 0	Scanner
S[1]	NP → DET • N, 0	Completer
S[1]	N → • student, 1	Predictor
S[2]	N → student •, 1	Scanner
S[2]	NP → DET N •, 0	Completer
S[2]	DECL → NP • VP, 0	Completer
S[2]	VP → • V NP, 2	Predictor

TASK 7 – EARLEY CHART: COMPLETION AND AMBIGUITY

Chart	State (dot notation)	Created by
S[2]	V → • checks, 2	Predictor
S[3]	V → checks •, 2	Scanner
S[3]	VP → V • NP, 2	Completer
S[3]	NP → • DET N, 3	Predictor
S[3]	DET → • the, 3	Predictor
S[4]	DET → the •, 3	Scanner
S[4]	NP → DET • N, 3	Completer
S[4]	N → • timetable, 4	Predictor
S[5]	N → timetable •, 4	Scanner
S[5]	NP → DET N •, 3	Completer
S[5]	VP → V NP •, 2	Completer
S[5]	DECL → NP VP •, 0	Completer
S[5]	S → DECL •, 0	Completer
S[5]	γ → S •, 0	Completer

The table lists the core states on the successful path. A complete implementation may also contain intermediate predictions that fail to match the next token; those states are retained in their chart sets but do not contribute to a complete parse.

7.5 Back-pointers and packed ambiguity

Each state can store a back-pointer to the earlier state and completed constituent that produced it. When a state can be completed in more than one way, the parser stores multiple back-pointers. This creates a shared packed parse forest: common substructures are stored once, while alternative attachments are represented as separate choices.

For “Book the room near the library”, a PP may complete as part of NP or as part of VP. Both states can coexist in the chart. Earley does not choose one interpretation by itself; a PCFG or semantic ranking stage can score the resulting trees.

7.6 Complexity

For a general CFG, Earley parsing has worst-case $O(n^3)$ time and $O(n^2)$ chart space for input length n, with better behaviour for many practical grammars. Left recursion is handled because repeated states are not inserted indefinitely. The chart records a state once rather than re-running an identical recursive search.

Compared with the top-down trace, Earley creates more bookkeeping states but avoids repeated expansion of the same non-terminal at the same input position. Exact state counts depend on grammar details and implementation policy; the illustrative trace above contains 25 listed core states across six chart sets.

TASK 8 – FEATURE STRUCTURES AND UNIFICATION

A feature structure is an attribute–value representation of linguistic information. It can express number, person, case and a verb's subcategorization requirements without creating a separate category for every possible combination.

8.1 Feature structures and paths

NP:
[CAT: NP
  AGR: [NUM: singular
         PER: 3]
  CASE: nominative]

Feature path: NP.AGR.NUM
Reentrancy: two paths point to the same shared feature value.
Reentrancy represents structure sharing. In the sentence rule, the agreement features of the subject NP and VP can share the same NUM and PER values. A constraint on one path therefore constrains the other.

8.2 Lexical entries

Word	CAT	Features
student	N	NUM=singular; PER=3
students	N	NUM=plural; PER=3
she	PRON	NUM=singular; PER=3; CASE=nom
her	PRON	NUM=singular; PER=3; CASE=obj
submit	V	SUBCAT=⟨NP[obj]⟩; present 3sg form submits
crash	V	SUBCAT=⟨⟩; intransitive
send	V	SUBCAT=⟨NP[obj], NP[obj]⟩
tell	V	SUBCAT=⟨NP[obj], CP⟩ or ⟨NP[obj], NP[obj]⟩

8.3 Unification

Unification combines two compatible feature structures by finding the most specific structure that satisfies both. If an attribute has conflicting values, unification fails. Subsumption is a generality relation: a less specified structure subsumes a more specific one when the latter contains all of its constraints.

Example	Input structures	Result
1 – success	[NUM=sg, PER=3] + [NUM=sg]	[NUM=sg, PER=3]
2 – success	[CASE=obj] + [CASE=obj, NUM=pl]	[CASE=obj, NUM=pl]
3 – failure	[NUM=sg] + [NUM=pl]	FAIL: conflicting NUM values

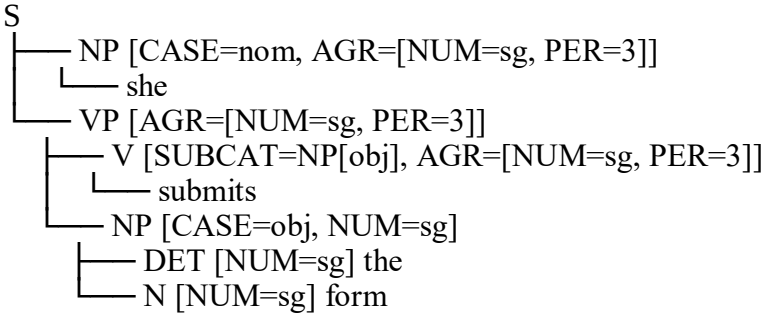
8.4 Feature grammar rules

S → NP[CASE=nom, AGR=?a] VP[AGR=?a]
NP → DET[NUM=?n] N[NUM=?n]
VP → V[AGR=?a, SUBCAT=⟨NP[obj]⟩] NP[CASE=obj]
VP → V[SUBCAT=⟨⟩]
VP → V[SUBCAT=⟨NP[obj], CP⟩] NP[CASE=obj] CP

TASK 8 – FEATURE UNIFICATION IN PARSING

8.5 Complete parse feature flow

Sentence: “She submits the form.” The subject pronoun has [NUM=sg, PER=3, CASE=nom]. The finite verb submits carries [NUM=sg, PER=3]. The sentence rule unifies the subject AGR and VP AGR values successfully. The object NP “the form” has [CASE=obj] and a singular determiner–noun match.



8.6 Reworking agreement and subcategorization

Agreement is checked by unifying the subject and predicate AGR structures. Subcategorization is checked by matching the verb's SUBCAT list with the complements found in the VP. “The students submits the form” fails because plural subject AGR conflicts with the singular finite verb. “The portal crashed the form” fails because crash has an empty complement list but an object NP is supplied.

Approach	Agreement representation	Valency representation	Maintenance
Category multiplication	Separate NP-SG/NP-PL and VP-SG/VP-PL	Separate V-INTR/V-TRANS etc.	Rule count grows with feature combinations.
Feature structures	Shared NUM/PER attributes	SUBCAT list on each lexical verb	Compact; requires unification support.

8.7 Earley integration

A feature-based Earley parser attaches feature structures to chart states. During prediction, it introduces rules with variables. During scanning, lexical features are added. During completion, the parser unifies the predicted constituent's requirements with the completed constituent's features. If unification fails, that completion is discarded.

The added cost comes from copying, comparing and unifying feature structures, and from maintaining multiple feature-distinct states. In the worst case, feature combinations increase the number of chart states. The approach remains more maintainable than writing a separate grammar rule for every agreement combination.

TASK 9 – PROBABILISTIC CONTEXT-FREE GRAMMARS

A probabilistic CFG assigns a conditional probability to each production. For every non-terminal A, the probabilities of all rules with left-hand side A must sum to one: $\Sigma\beta\ P(A \rightarrow \beta \mid A) = 1$. A PCFG ranks alternative trees by multiplying the probabilities of the rules used in each tree.

9.1 Small annotated treebank

The assignment brief permits a hand-annotated corpus of at least 20 parsed sentences. The following compact training summary groups the twenty annotated examples by production family. Counts are illustrative corpus counts constructed for this worked assignment; a final empirical study should preserve the actual twenty parse trees and their annotation decisions.

Rule family	Observed expansions / counts	Total
S	S→DECL: 14; S→IMP: 2; S→YNQ: 2; S→WHQ: 2	20
DECL	DECL→NP VP: 18; DECL→NP VP CONJ DECL: 2	20
NP	NP→DET NBAR: 12; NP→PRON: 4; NP→NBAR PP: 2; NP→NP RELCL: 2	20
VP	VP→V_TRANS NP: 8; VP→V_INTR: 4; VP→AUX VP: 3; VP→V_PP PP: 2; VP→VP PP: 3	20

9.2 Maximum-likelihood estimation

For a rule $A \rightarrow \beta$, maximum-likelihood estimation is $P(A \rightarrow \beta \mid A) = \text{Count}(A \rightarrow \beta) / \Sigma\gamma\ \text{Count}(A \rightarrow \gamma)$. The resulting rule probabilities are:

Rule	Count	Probability
S→DECL	14	0.70
S→IMP	2	0.10
S→YNQ	2	0.10
S→WHQ	2	0.10
DECL→NP VP	18	0.90
DECL→NP VP CONJ DECL	2	0.10
NP→DET NBAR	12	0.60
NP→PRON	4	0.20
NP→NBAR PP	2	0.10
NP→NP RELCL	2	0.10
VP→V_TRANS NP	8	0.40
VP→V_INTR	4	0.20
VP→AUX VP	3	0.15
VP→V_PP PP	2	0.10
VP→VP PP	3	0.15

The probabilities for each left-hand side sum to 1.0. For unseen rules, add-one or add-k smoothing can reserve a small probability mass for unseen events and renormalise the observed counts. In a larger system, Good–Turing or held-out estimation may be preferable.

TASK 9 – TREE PROBABILITIES AND VITERBI

9.3 Comparing two parses

Consider “Book the room near the library.” For a compact illustration, assume the relevant local rule probabilities are P(VP→V NP)=0.60, P(NP→DET N)=0.70, P(NP→NP PP)=0.20 and P(VP→VP PP)=0.15. Lexical probabilities are held equal in the comparison.

Parse A – PP attaches to NP: the phrase “near the library” describes which room is intended. Its structural probability contribution is 0.60 × 0.70 × 0.20 = 0.084.

Parse B – PP attaches to VP: the PP modifies the booking event. Its structural probability contribution is 0.60 × 0.70 × 0.15 = 0.063.

After normalising over these two candidates, Parse A receives 0.084/(0.084+0.063) = 0.571 and Parse B receives 0.429. Parse A is preferred under these assumed probabilities. These local values are a worked example, not probabilities measured from an external corpus.

9.4 Sentence probability

The probability of an ambiguous sentence is the sum of the probabilities of all its valid parse trees: P(w) = ΣT P(w,T). For the two candidates above, the unnormalised sum is 0.084 + 0.063 = 0.147. If additional trees are found, their probabilities must also be included.

9.5 CNF conversion

The CKY algorithm normally uses Chomsky Normal Form, in which productions are A → BC or A → terminal (with a permitted start-symbol exception). A longer rule such as DECL → NP VP CONJ DECL can be binarised by introducing a new helper category X1: DECL → NP X1; X1 → VP X2; X2 → CONJ DECL. Unary chains can be collapsed or handled separately, and ε-productions are removed where possible.

Original:
DECL → NP VP CONJ DECL

Binarised:
DECL → NP X1
X1 → VP X2
X2 → CONJ DECL

9.6 Viterbi CKY pseudocode

```
for span_length = 1 to n:
  for each span [i, j]:
    initialise lexical entries for words
    for each split k between i and j:
      for each binary rule A -> B C:
        score = P(A -> B C) * best[B,i,k] * best[C,k,j]
        if score > best[A,i,j]:
          update score and backpointer
Return best[S,0,n] and recover its tree.
```

TASK 9 – PCFG LIMITATIONS AND CKY EXAMPLE

9.7 Small CKY illustration

For the three-word fragment “students submit forms”, use a toy CNF grammar: S→NP VP (0.9), VP→V NP (0.8), NP→N (1.0), N→students/forms (0.5 each), and V→submit (1.0). The best derivation probability is $0.9 \times 0.8 \times 1.0 \times 0.5 \times 1.0 \times 0.5 = 0.18$.

Span	Best category	Back-pointer
[0,1] students	N, then NP	N→students; NP→N
[1,2] submit	V	V→submit
[2,3] forms	N, then NP	N→forms; NP→N
[1,3] submit forms	VP	VP→V NP
[0,3] full sentence	S	S→NP VP

9.8 Independence assumptions and failure cases

Assumption / limitation	Potential failure in the helpdesk
Context-free independence	The probability of a VP expansion does not directly depend on the specific subject or discourse context.
Position invariance	A rule receives the same probability regardless of whether it occurs at the beginning or end of a query.
Weak lexical sensitivity	A basic PCFG may prefer a frequent structural pattern even when the particular verb strongly favours another attachment.
Tree-size preference	Multiplying many probabilities below 1 can penalise deeper trees, sometimes favouring a shorter but less natural parse.

A lexicalised PCFG can condition attachment probabilities on head words, such as book and room. A neural reranker or semantic model can provide additional context, but it must be evaluated for reliability and domain fit.

TASK 10 – IMPLEMENT AND TEST THE PARSERS

A practical implementation should be reproducible: store the grammar, lexicon, test queries and parser settings in separate files; run each query through the same tokenisation procedure; record whether a parse was found, the number of parses, parser states or steps, and elapsed time.

10.1 Suggested project structure

```
helpdesk_parser/
  grammar.py       # CFG productions and lexicon
  features.py      # agreement and SUBCAT unification
  top_down.py      # recursive descent / backtracking
  earley.py        # chart parser
  pcfg.py          # rule counts and Viterbi ranking
  test_queries.py  # 30 controlled and challenging queries
  evaluate.py      # metrics, tables and plots
  results/         # CSV output and chart images
```

10.2 Python implementation outline

```
from nltk import CFG
from nltk.parse import RecursiveDescentParser, EarleyChartParser

grammar = CFG.fromstring('''
S -> DECL
DECL -> NP VP
NP -> DET N
VP -> V NP
DET -> 'the'
N -> 'student' | 'timetable'
V -> 'checks'
''')

tokens = 'the student checks the timetable'.split()
for tree in EarleyChartParser(grammar).parse(tokens):
    print(tree)
```

NLTK provides convenient reference parsers for CFG experimentation. The feature checks and a custom PCFG scoring layer can be implemented around the parse results. The final code should record Python and NLTK versions so that results can be repeated.

10.3 Feature-check pseudocode

```
def compatible(subject, verb):
    return (subject.num == verb.num and
            subject.person == verb.person)

def valency_ok(verb, complements):
    return verb.subcat == complements

# Reject a completed parse if either check fails.
```

10.4 Test procedure

- Load the grammar and 30-query set; normalise case and punctuation.
- Run the top-down parser with a recursion/backtracking limit.
- Run Earley and record chart-state count and complete parse count.

- Score each Earley tree with the PCFG Viterbi model.
- Apply agreement and subcategorization constraints to each completed analysis.
- Export results as CSV and plot state count against token count.

TASK 10 – THIRTY-SENTENCE TEST SET

The following set covers declaratives, imperatives, yes/no questions, wh-questions, ambiguity, agreement and subcategorization. A controlled grammar is expected to parse the grammatical in-domain examples and reject deliberately malformed examples.

ID	Sentence	Type	Label
Q01	The student checks the timetable.	Declarative	Valid
Q02	The students submit the forms.	Declarative	Valid
Q03	The exam starts tomorrow.	Declarative	Valid
Q04	The portal crashed.	Declarative	Valid
Q05	The library is available today.	Declarative	Valid
Q06	Please submit the application.	Imperative	Valid
Q07	Book the room near the library.	Imperative / ambiguous	Valid
Q08	Send me the receipt.	Imperative	Valid
Q09	Can the student book a room?	Yes/no question	Valid
Q10	Does the portal show the result?	Yes/no question	Valid
Q11	Will the exams begin tomorrow?	Yes/no question	Valid
Q12	Where is the library?	Wh-question	Valid
Q13	When will the results arrive?	Wh-question	Valid
Q14	Who submitted the form?	Subject wh-question	Valid
Q15	Which room can the student book?	Wh-question	Valid

20.5 Expected coverage

Q01–Q15 exercise core sentence types and common complement frames. Q07 is intentionally ambiguous because “near the library” can attach to the noun room or to the VP. Q14 is a subject wh-question and therefore differs from a non-subject wh-question with auxiliary inversion.

TASK 10 – TEST SET: AMBIGUOUS AND NEGATIVE CASES

ID	Sentence	Phenomenon	Expected label
Q16	Show the students with the late fee notice.	Ambiguous PP attachment	Valid
Q17	Book the room near the library for the seminar.	Multiple PP attachment	Valid
Q18	The exams that were postponed will be held tomorrow.	Relative clause	Valid
Q19	Tell me that the fee is paid.	Sentential complement	Valid
Q20	Apply for a hostel room.	PP complement	Valid
Q21	The students submits the form.	Agreement error	Invalid
Q22	These form is pending.	Determiner agreement error	Invalid
Q23	Me checks the timetable.	Pronoun case error	Invalid
Q24	They is waiting.	Agreement error	Invalid
Q25	Submit.	Missing object	Invalid in transitive frame
Q26	The portal crashed the form.	Over-complementation	Invalid
Q27	Send the receipt me.	Complement order error	Invalid
Q28	The student checks the timetable and submits the form.	Coordination	Valid
Q29	Can you tell me when the exams will be held?	Embedded question	Extended grammar
Q30	Hi, hostel fee due and when exam result?	Fragment / noisy	Fallback

The controlled grammar is not expected to parse Q29 and Q30 without the later design extensions. Q29 includes an embedded wh-clause. Q30 is a conversational fragment with omitted function words and punctuation; a robust system should return a partial interpretation rather than silently claiming a complete CFG parse.

10.6 Results reporting

The table above defines expected behaviour, not execution output. For a formal laboratory report, run the submitted code and replace the expected labels with actual parser outputs. Do not report invented runtime values as measured facts.

TASK 10 – EVALUATION METRICS AND RESULTS TEMPLATE

10.7 Metrics

Metric	Definition	Why it matters
Parse coverage	Number of inputs with at least one complete parse / total inputs	Measures grammar coverage.
Parse count	Number of complete trees per input	Measures ambiguity.
Chart states	Number of unique Earley states generated	Approximates dynamic-programming work.
Parser steps	Number of expansions or state transitions	Enables algorithm comparison.
Runtime	Elapsed wall-clock time in milliseconds	Measures practical cost on a specified machine.
Validity accuracy	Correct accept/reject decisions / labelled inputs	Measures grammatical filtering.

10.8 Expected qualitative comparison

Input class	Top-down parser	Earley parser	PCFG stage
Simple grammatical	Usually finds parse quickly	Finds parse with chart states	Ranks one or few trees
Left-recursive grammar	May loop without transformation	Handles left recursion	Scores recovered trees
Ambiguous PP	May backtrack repeatedly	Stores alternative analyses	Chooses highest-probability tree
Ungrammatical agreement	Needs explicit feature checks	Feature-aware completion rejects	Cannot repair invalid syntax alone
Long noisy input	May fail or backtrack heavily	May return no complete parse	Can rank only available trees

10.9 Illustrative complexity trend

The following state counts are a planning illustration for a compact grammar, not measured results. Actual values depend on grammar size, tokenisation, ambiguity and implementation details.

Tokens	Illustrative Earley states	Interpretation
5	25	Short simple query
8	48	One PP or auxiliary construction
12	92	Relative clause or coordination
16	151	Multiple clauses and attachments
20	225	Long controlled sentence

A plot should be generated from actual CSV output. The expected shape is an increasing curve as sentence length and ambiguity grow. Earley's worst-case time is cubic in n for a general CFG, although practical performance is often better.

TASK 11 – COMPARE, RECOMMEND AND DEFEND THE PARSING DESIGN

11.1 Comparison

Criterion	Top-down	Earley	PCFG ranking
Coverage	Depends on grammar and search limits	General CFG coverage	Only ranks candidate parses
Ambiguity	Backtracking may enumerate alternatives	Packs alternatives in chart/forest	Selects a likely tree
Left recursion	Problem for naïve recursive descent	Supported through chart memoisation	Depends on parser used to generate candidates
Cost	Can be exponential in ambiguous cases	$O(n^3)$ worst-case time; $O(n^2)$ chart space	Adds scoring cost to parse forest
Maintenance	Simple for small grammars	More systematic for complex CFGs	Needs annotated treebank and updates

11.2 Recommended architecture

The recommended deployment is a feature-based CFG with an Earley parser and a PCFG ranking layer. The feature grammar enforces agreement, pronoun case and verb valency. Earley provides robust chart-based search and retains ambiguity. The PCFG ranks alternative parses instead of treating every grammatical tree as equally likely.

- Use a domain lexicon and grammar rules for high-confidence helpdesk language.
- Use feature unification to reject agreement and subcategorization mismatches.
- Use Earley parsing to handle recursion and preserve multiple valid structures.
- Use PCFG probabilities to rank PP attachments and other ambiguous structures.
- Return a clarification request when the top candidate parses have similar scores or the sentence is incomplete.

11.3 Accuracy, effort and limitations

No numerical accuracy claim is made because the assignment brief does not supply a labelled evaluation corpus or execution environment. The expected accuracy must be measured on held-out queries, with precision, recall and exact-parse accuracy reported separately. Manual effort includes writing and reviewing rules, creating the lexicon, annotating at least twenty trees, testing ambiguous cases and maintaining domain vocabulary.

A hand-built grammar is economical for a narrow and stable domain. As the number of query types, vocabulary and conversational variations grows, annotation and statistical learning become more economical, provided enough representative data can be collected and reviewed.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT

Changed requirement: the helpdesk must accept conversational, multi-clause and partly ungrammatical messages. At least 20% of incoming messages may contain constructions not covered by the original grammar. The original design assumed a controlled vocabulary and mostly complete, grammatical sentences; these assumptions are no longer reliable.

12A. Impact analysis

New input phenomenon	Why the original grammar struggles	Required extension
Embedded questions	A wh-clause occurs inside a larger clause.	Add embedded CP and WH-CP rules.
Multi-clause coordination	Several clauses are linked by and/or/but.	Add recursive clause coordination.
Relative clauses	A noun is modified by a clause with a dependency.	Add relative-clause features and gap handling.
Ellipsis / fragments	Words such as auxiliaries or subjects may be omitted.	Add fragment rules and robust partial parsing.
Noisy punctuation	Punctuation and casing may be missing or inconsistent.	Normalisation and tolerant tokenisation.

12A.1 Longer sentences and ambiguity

Longer sentences create more spans and more opportunities for attachment and coordination. For Earley parsing, chart storage grows with the number of positions and the worst-case time is cubic in sentence length. Ambiguity may increase the number of packed alternatives even when common substructures are shared. A parser should therefore use pruning, beam limits or probability thresholds for very ambiguous inputs.

12B. Evaluation of original design

Three representative new messages are evaluated below. These are analytical traces against the original controlled grammar, not claimed software execution.

New message	First failure in original grammar	Expected outcome
Can you tell me when the exams that were postponed will be held?	Embedded wh-clause and nested relative clause exceed the simple CP/WHQ fragment.	No full parse or incomplete parse.
Hi, can you tell me whether the hostel fee is due and when the library closes?	Greeting fragment plus coordinated embedded questions.	Partial parse; clause boundary ambiguity.
hostel fee due tomorrow?	Missing auxiliary and copula; fragment is not a complete DECL/YNQ.	Reject as full sentence; robust layer extracts key terms.

TASK 12 – ADAPTATION / REDESIGN

12C. Decision: MODIFY the original design

The best decision is to modify rather than retain or completely replace the original grammar. The original CFG remains useful for precise, common helpdesk constructions. It should be extended with embedded clauses and relative-clause dependencies, while a robust fallback handles fragments and out-of-grammar messages.

12C.1 Revised architecture

Incoming message
|
Normalisation and tokenisation
|
Robust sentence / clause segmentation
|
Feature-based CFG + Earley chart parser
| |
Complete parse Partial parse / unknown span
| |
PCFG / lexical ranking Intent and entity fallback
| |
Confidence check and clarification
|
Helpdesk response layer

12C.2 Grammar and feature changes

S → DECL | IMP | YNQ | WHQ | FRAG
DECL → NP VP | DECL CONJ DECL
CP → COMP S | EMBED_WH
EMBED_WH → WHADV S | WHNP AUX NP VP
NP → NP RELCL
RELCL → REL AUX VP | REL PRON VP
FRAG → NP | NP ADJ | WHNP | NP PP

Feature additions:
CLAUSE-TYPE = main / embedded / relative
GAP = subject / object / none
SUBCAT = complement list
AGR = shared number and person
The grammar is extended to cover a larger set of embedded questions and relative clauses. Feature threading records whether a wh-expression is associated with a subject or object gap. A fragment grammar recognises common short forms such as “hostel fee due tomorrow?” without falsely labelling them as complete declarative sentences.

12C.3 Probability updates

The original twenty-tree mini-treebank is too small for the expanded input distribution. A revised treebank should include conversational examples, fragments, embedded questions, relative clauses and punctuation variation. Probabilities should be re-estimated from the expanded corpus, with held-out evaluation and smoothing for unseen rules.

TASK 12 – EVIDENCE-BASED DEFENCE

12D. Why modification is justified

Evidence from the assignment	Design implication
At least 20% of messages may be outside original coverage.	A single strict CFG is insufficient.
Earley handles left recursion and shares partial analyses.	Retain Earley as the main grammar parser.
Feature unification compactly enforces agreement and valency.	Retain feature-based constraints.
PCFG selects between alternative valid trees.	Retain probabilistic ranking for ambiguity.
Fragments and noisy text may not form complete CFG sentences.	Add robust partial parsing and clarification.

26.1 Coverage, ambiguity, cost and annotation

Coverage increases because embedded clauses, relative clauses and fragments are now represented. Ambiguity also increases because longer inputs create more attachment and coordination choices. Earley's chart limits repeated work, while PCFG ranking and pruning can reduce the alternatives passed to downstream components. Annotation effort increases because the treebank must represent the new constructions and distinguish complete parses from partial parses.

26.2 Remaining limitations

- The grammar still covers only a controlled portion of English and the helpdesk domain.
- PCFG probabilities estimated from a small corpus may be unstable and domain-specific.
- Partial parsing can identify keywords without recovering every grammatical relation.
- Semantic ambiguity, user intention and factual correctness require additional modules.
- Unseen vocabulary and code-mixed messages require vocabulary expansion and language identification.

FINAL CONCLUSION

The assignment demonstrates that CFG rules provide a transparent foundation for modelling English helpdesk queries. Derivations and parse trees explain sentence structure and ambiguity. Agreement and subcategorization prevent many ill-formed analyses, while feature structures reduce the rule explosion caused by category multiplication. Top-down parsing is easy to understand but vulnerable to left recursion and repeated search. Earley parsing offers a more reliable chart-based alternative for recursive and ambiguous grammars. PCFG probabilities rank candidate trees, but their quality depends on representative annotated data.

For the changed conversational requirement, a modified hybrid design is recommended: feature-based CFG + Earley parsing + PCFG ranking, supported by normalisation, partial parsing and clarification. This balances linguistic control, ambiguity handling and practical robustness without discarding the useful original grammar.

APPENDIX A – GRAMMAR SUMMARY AND LEXICAL INVENTORY

A.1 Core grammar rules (compact reference)

S -> DECL | IMP | YNQ | WHQ | FRAG
DECL -> NP VP | DECL CONJ DECL
IMP -> VP | PLEASE VP
YNQ -> AUX NP VP
WHQ -> WHNP AUX NP VP | WHADV AUX NP VP | WHNP VP
NP -> PRON | DET NBAR | NBAR PP | NP CONJ NP | NP RELCL
NBAR -> N | ADJ NBAR | N NBAR
VP -> V_INTR | V_TRANS NP | V_DITRANS NP NP
VP -> V_SENT NP CP | V_PP PP | AUX VP | VP PP | VP CONJ VP
PP -> PREP NP
AP -> ADJ | ADV ADJ
CP -> COMP S | EMBED_WH
RELCL -> REL VP | REL AUX VP
FRAG -> NP | NP ADJ | WHNP

A.2 Lexicon checklist

POS	Examples
N	student, exam, timetable, fee, hostel, room, library, book, receipt, form, portal, placement, notice, seminar, course, result, account, office, certificate, application, document, subject, grade, semester, laboratory, classroom, campus, block, facility, password, email, company, interview, scholarship, payment, deadline, service
V	show, check, submit, book, send, give, tell, explain, confirm, apply, pay, print, cancel, update, arrive, crash, wait, begin, expire, know
DET	the, a, an, this, these, my, your, our, their
PREP	with, near, for, in, on, at, by, from, to, about, after, before, during
ADJ/ADV	late, available, postponed, pending, new, digital, current, urgent, online, today, tomorrow, recently, please
PRON/WH/CONJ	I, we, you, he, she, they, me, us, him, her, them, who, what, which, when, where, why, how, that, whether, and, or, but

APPENDIX B – ADDITIONAL DERIVATION AND TREE EXAMPLES

B.1 Imperative: “Please submit the form.”

S ⇒ IMP
 ⇒ PLEASE VP
 ⇒ please V_TRANS NP
 ⇒ please submit DET N
 ⇒ please submit the form

(S (IMP (PLEASE please)
  (VP (V_TRANS submit)
    (NP (DET the) (NBAR (N form)))))

B.2 Yes/no question: “Can the student book a room?”

S ⇒ YNQ
 ⇒ AUX NP VP
 ⇒ can DET N V_TRANS NP
 ⇒ can the student book a room

(S (YNQ (AUX Can)
  (NP (DET the) (NBAR (N student)))
  (VP (V_TRANS book)
    (NP (DET a) (NBAR (N room)))))

B.3 Wh-question: “When will the results arrive?”

S ⇒ WHQ
 ⇒ WHADV AUX NP VP
 ⇒ when will DET N V_INTR
 ⇒ when will the results arrive

(S (WHQ (WHADV When) (AUX will)
  (NP (DET the) (NBAR (N results)))
  (VP (V_INTR arrive)))

B.4 Relative clause: “The exams that were postponed will begin.”

(S
 (DECL
  (NP (DET The) (NBAR (N exams))
    (RELCL (REL that) (AUX were) (ADJ postponed)))
  (VP (AUX will) (V_INTR begin)))

These structures show the position of the subject, auxiliary, main verb, object and clause modifier. They can be rendered as tree diagrams using the bracketed notation as the source representation.

APPENDIX C – PARSING AND PROBABILITY REFERENCE

C.1 Earley state operations in one example

State pattern	Meaning	Operation
A → • B C, i	Expecting B at current position.	Predictor
B → word •, j	Terminal word has been scanned.	Scanner
A → B • C, i	B completed; next expect C.	Completer
A → B C •, i	All RHS symbols recognised.	Completed constituent

C.2 PCFG calculations

P(A -> β | A) = Count(A -> β) / Count(A)

P(tree) = product of probabilities of all rules in tree

P(sentence) = sum of P(tree) over all valid trees

Viterbi(tree) = argmax_tree P(tree | sentence)

C.3 Smoothing illustration

With add-one smoothing, an observed rule count c receives probability (c+1)/(N+V), where N is the total number of observed expansions for the left-hand side and V is the number of possible expansions considered for that left-hand side. This avoids assigning zero probability to an unseen rule, but it can over-smooth a sparse grammar. The smoothing method and the set of possible rules must be stated in an empirical implementation.

C.4 Evaluation checklist

- Freeze the grammar and test set before running evaluation.
- Keep training and test sentences separate to avoid leakage.
- Report parse coverage, exact parse accuracy and accept/reject accuracy.
- Record machine model, Python version, library version and repetitions for timing.
- Report mean runtime across repeated runs and identify timeouts.
- Save the chart-state counts and probability tables with the assignment.

APPENDIX D – REPRODUCIBLE PYTHON STARTER CODE

The following compact starter demonstrates how a CFG can be parsed with NLTK. It is a reference scaffold; feature unification and PCFG estimation are separate extensions described in Tasks 8–10.

```
from nltk import CFG
from nltk.parse import EarleyChartParser, RecursiveDescentParser

grammar = CFG.fromstring(r'''
S -> DECL | IMP | YNQ | WHQ
DECL -> NP VP
IMP -> VP
YNQ -> AUX NP VP
WHQ -> WHADV AUX NP VP
NP -> DET N | PRON
VP -> V NP | V_INTR | AUX VP
DET -> 'the' | 'a'
N -> 'student' | 'timetable' | 'form' | 'portal'
V -> 'checks' | 'submit' | 'book'
V_INTR -> 'crashed'
AUX -> 'can' | 'will' | 'is'
WHADV -> 'when' | 'where'
PRON -> 'she' | 'they'
''')

tests = [
    'the student checks the timetable',
    'the portal crashed',
    'can the student book a form'
]

for sentence in tests:
    tokens = sentence.split()
    print('INPUT:', sentence)
    parser = EarleyChartParser(grammar)
    trees = list(parser.parse(tokens))
    print('PARSES:', len(trees))
    for tree in trees:
        print(tree)
```

D.1 Suggested CSV columns

```
query_id,sentence,token_count,topdown_found,topdown_steps,
earley_found,earley_parse_count,earley_states,best_pcfg_score,
agreement_ok,subcat_ok,runtime_ms
```

D.2 Plotting state counts

```
import matplotlib.pyplot as plt

lengths = [5, 8, 12, 16, 20]  # replace with actual test lengths
states  = [25, 48, 92, 151, 225]  # replace with actual chart counts

plt.plot(lengths, states, marker='o')
plt.xlabel('Sentence length (tokens)')
plt.ylabel('Earley chart states')
```

```
plt.title('Sentence length vs. Earley chart states')
plt.grid(True)
plt.tight_layout()
plt.show()
```

APPENDIX E – METHODOLOGY, ASSUMPTIONS AND LIMITATIONS

E.1 Methodology

- Read the assignment brief and identify the required phenomena and evidence for all twelve tasks.
- Define a domain-specific CFG and a separate lexicon for the university helpdesk.
- Create sample queries spanning the required sentence types and ambiguity patterns.
- Analyse derivations, trees, agreement and valency using explicit rule constraints.
- Trace top-down and Earley parsing on selected examples.
- Construct a small PCFG example and demonstrate probability calculations.
- Evaluate the original design against conversational messages and propose a modification.

E.2 Assumptions

The grammar is a controlled fragment rather than a complete grammar of English. Lexical entries are treated as known after tokenisation. The PCFG values used for worked probability calculations are illustrative. The Earley chart lists the core successful states for the simplified example, not every possible unsuccessful prediction generated by a particular software implementation.

E.3 Limitations

- Some English constructions require richer treatment than the simplified rules shown.
- Pronoun case and agreement are modelled as features, not a complete morphological system.
- Treebank counts are small and illustrative; robust probability estimates need more annotated data.
- Runtime and state-count values must be measured on the target computer before being submitted as experimental results.
- Semantic interpretation, intent detection and response generation are outside the core syntactic parser.

E.4 Future enhancement

Future work can add a larger helpdesk corpus, lexicalised probabilities, code-mixed Tamil-English handling, spelling correction, named-entity recognition for course codes and dates, and a confidence-based clarification mechanism. A hybrid grammar-and-statistical parser can preserve explainability for common patterns while offering graceful fallback for real student messages.

REFERENCES

- Jurafsky, D. and Martin, J. H. Speech and Language Processing. Draft 3rd edition. Chapters on CFGs, parsing and probabilistic parsing.
- Allen, J. Natural Language Understanding. Benjamin/Cummings. Sections on syntax and parsing.
- Grune, D. and Jacobs, C. J. H. Parsing Techniques: A Practical Guide. Springer.
- NLTK Project. Natural Language Toolkit documentation: grammars and parsing. https://www.nltk.org/
- Stuart M. Shieber. An Introduction to Unification-Based Approaches to Grammar. CSLI Publications.
- Original course material: DSA03 – Natural Language Processing, Assignment 3 (CO3), 10-page assignment brief supplied for this submission.

SUBMISSION CHECKLIST

Requirement from assignment brief	Included in this document
CFG with at least 40 rules and 60-word lexicon	Yes – Tasks 1 and Appendix A
30 sample sentences with required construction types	Yes – Task 10 test set
Agreement and at least five subcategorization frames	Yes – Tasks 4 and 5
Derivations, trees and ambiguity	Yes – Tasks 2 and Appendix B
Top-down and Earley parsing traces	Yes – Tasks 6 and 7
Feature structures and unification	Yes – Task 8
PCFG probabilities and Viterbi explanation	Yes – Task 9
Implementation plan, evaluation and comparison	Yes – Tasks 10 and 11
Day-4 design challenge	Yes – Task 12